14. Design of Image thresholding using Xilinx Vivado HLS (Gray to Binary image conversion)

Aim

To design and implement an image thresholding system using Xilinx Vivado HLS, where a C++ based thresholding algorithm is converted into RTL hardware, verified using a Verilog testbench, and the processed image output is visualized using Python.

Software Required

- Xilinx Vivado Design Suite
- Vivado HLS
- Vivado Simulator (XSim)
- Verilog HDL
- Python

Procedure

1. **C++ Algorithm Design**
    - A C++ function was written using the ap_uint<8> data type to process grayscale image pixels.
    - A fixed threshold value of 128 was used to convert grayscale pixels into binary values.
    - Pixels greater than or equal to the threshold were mapped to 255, and others to 0.
2. **High-Level Synthesis**
    - The C++ program was synthesized using Vivado HLS.
    - ap_none interfaces were used to generate a simple data-only RTL module without clock or control signals.
    - Pipelining was applied to improve throughput.
3. **RTL Generation**
    - Vivado HLS converted the C++ design into Verilog RTL.
    - The generated RTL represented a comparator-based thresholding circuit.
4. **Verilog Testbench Verification**
    - A Verilog testbench was written to apply pixel values to the RTL module.
    - The output binary pixel values were observed and verified through simulation.
5. **Image Input Generation**
    - A grayscale image was converted into a sequence of pixel values using Python.
    - The pixel sequence was provided as input to the Verilog testbench.

6. **Output Visualization**
    - The RTL output was captured into a text file during simulation.
    - Python was used to reconstruct and visualize the output binary image.

Python program to convert the 256x256 image to 1D form suitable for verilog module

```
from PIL import Image
import numpy as np
img = Image.open("input_image.png").convert("L")
img = img.resize((256, 256))
img_array = np.array(img, dtype=np.uint8)
with open("image_in.txt", "w") as f:
    for pixel in img_array.flatten():
        f.write(str(int(pixel)) + "\n")
```

C++ program for image thresholding

```
#include <ap_int.h>
#define THRESHOLD 128
ap_uint<8> image_threshold(ap_uint<8> pixel_in) {
#pragma HLS INTERFACE ap_none port=pixel_in
#pragma HLS INTERFACE ap_none port=return
#pragma HLS PIPELINE
    ap_uint<8> pixel_out;
    if (pixel_in >= THRESHOLD)
        pixel_out = 255;
    else
        pixel_out = 0;
    return pixel_out;
}
```

Testbench program

```
`timescale 1ns/1ps
module tb_image_threshold;
```

```
 parameter WIDTH  = 256;
 parameter HEIGHT = 256;
 parameter SIZE   = WIDTH * HEIGHT;
 reg  [7:0] gray_pixel;
 wire [7:0] binary_pixel;
 reg [7:0] image_mem [0:SIZE-1];
 integer i;
 integer infile, outfile;
image_threshold UUT (
    .ap_start(1'b1),
    .ap_done(),
    .ap_idle(),
    .ap_ready(),
    .pixel_in_V(gray_pixel),
    .ap_return(binary_pixel)
 );
 initial begin
  infile  = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\image_in.txt", "r");
  outfile = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\image_out.txt", "w");
  for (i = 0; i < SIZE; i = i + 1) begin
    if (!$feof(infile))
      $fscanf(infile, "%d\n", image_mem[i]);
    else begin
      $display("ERROR: Insufficient image data.");
      $finish;
    end
  end
  for (i = 0; i < SIZE; i = i + 1) begin
    gray_pixel = image_mem[i];
    #1;
    $fdisplay(outfile, "%d", binary_pixel);
```

```
    end
    $fclose(infile);
    $fclose(outfile);
    $finish;
  end
```

Python program to convert the 1D output data generated by verilog module to 256x256 image

```
import numpy as np
from PIL import Image
data = np.loadtxt("image_out.txt", dtype=np.uint8)
img = data.reshape((256,256))
Image.fromarray(img).save("binary_output.png")
```

Output

Gray image

Binary image

Result

The image thresholding system was successfully designed and verified using Xilinx Vivado HLS. The grayscale image pixels were correctly converted into binary values based on the predefined threshold. Simulation results matched the expected output, and the reconstructed image displayed clear foreground and background separation. This confirms the correct functionality of the HLS-based image thresholding design.